From Threat Intelligence to Detection: Knowledge-driven Enrichment and Template-based Rule Grounding for Automated Sigma Rule Generation

Sepehr Ghaffarzadegan, Boubakr Nour, Makan Pourzandi, Mourad Debbabi, and Chadi Assi

arXiv:2608.19011v1 [cs.CR] 19 Aug 2026

***Abstract*—Mechanisms for dynamically converting cyber threat intelligence (CTI) into actionable detection capabilities are necessary due to the rapid evolution of Advanced Persistent Threats (APTs). Sigma rules are an essential part of contemporary threat detection workflows because they offer a platform-independent framework for expressing detection logic that can be converted into particular queries across SIEM systems. Conventional techniques for manually crafting Sigma rules are prone to mistakes, and necessitate extensive knowledge, which restricts their scalability. Although there are open-source and industry-maintained Sigma rule repositories, they often fail to keep pace with emerging threats and require frequent customization to fit diverse operational environments. This emphasizes the necessity of dynamic rule generation that is adapted to evolving attack techniques as well as particular use cases. In this work, we design AUTOSIGMA, an automated solution for transforming unstructured CTI reports into relevant Sigma rules. Rather than relying solely on language models, AUTOSIGMA leverages a structured knowledge base to enrich partial inputs, matches the enriched content against a repository of existing Sigma rules, and then employs an LLM-as-a-Judge mechanism to iteratively validate the rules. By combining knowledge-driven enrichment, template-based rule grounding, and a multi-stage solution, AUTOSIGMA enables accurate, context-aware, and relevant rule generation. Evaluations across multiple real-world APT reports and multiple security blogs demonstrate that AUTOSIGMA outperforms alternative solutions and LLM models in rule validity, rule relevancy, MITRE ATT&CK technique coverage, and robustness to input quality.**

AUTOSIGMA's Demo: https://youtu.be/iSr6IurQ6BM

***Index Terms*—Rule Generation, Sigma Rules, Security Automation, Cyber Threat Intelligence**

I. INTRODUCTION

Advanced Persistent Threats (APTs) represent a growing menace to organizations and critical infrastructure. In practice, defenders rely heavily on detailed Cyber Threat Intelligence (CTI) reports from industry leaders (*e.g.,* Mandiant [1], CrowdStrike [2]). These reports document the tactics, techniques, procedures (TTPs), malware, and infrastructure used by attackers, providing invaluable context for defense. Despite this wealth of information, CTI reports are typically published as unstructured narratives rather than machine-readable or actionable outputs ready-to-use by practitioners. The widespread use of detection rules in industry further highlights the need for adaptable and relevant detection logic. In practice, security teams often need to write new detection rules (*e.g.,* Sigma) manually to support proactive threat hunting and to address diverse operational contexts. Translating these unstructured threat descriptions into concrete detection content (*e.g.,* Sigma rules) is thus a bottleneck in a proactive defense stance when the Sigma rules can be generated at run-time from internal and external threat intelligence reports [3]. Existing CTI reports often emphasize known indicators of compromise (IoCs) which can be useful for detection but have limited longevity. To counter evolving threats, analysts must extract deeper patterns (*e.g.,* attack sequences or behaviors) from narrative text beyond transient indicators. The Sigma format has become a common schema for writing generic detection rules, therefore a good candidate to extract these deeper patterns. At the same time, crafting Sigma rules manually is labor-intensive and error-prone. Automating this pipeline from raw CTI text to relevant Sigma rules remains an open challenge.

Prior efforts have tackled parts of this problem. For example, TTPDRILL [4] uses an ontology-based approach to parse CTI text, automatically extracting threat actions and mapping them to MITRE ATT&CK techniques. THREATRAPTOR [5] builds on this idea by linking extracted indicators and actions into a threat behavior graph that can be searched against system logs. Similarly, LADDER [6] aggregates attack patterns across multiple CTI sources to capture multi-stage intrusions. These systems demonstrate the value of structuring CTI, but they fall behind in producing ready-to-deploy detection rules. They often require well-structured input or significant manual curation, and they do not integrate with a rule repository to ground outputs in known detection logic. More recently, large language models (LLMs) have been applied to accelerate CTI to rule generation. Work in [7] and [8] review how LLMs and generative AI are transforming cybersecurity, including threat intelligence extraction and rule generation. These studies suggest LLMs can effectively interpret natural-language reports, but also highlight that automated rule synthesis remains immature.

Our Contribution: Fig. 1 illustrates how AUTOSIGMA helps and enhances the threat hunting process [9]. CTI reports, attack trends, and behavioral indicators drive analysis and

This work was made possible in part through the support of the National Cybersecurity Consortium and the Government of Canada. This work was also supported in part by Ericsson Research and the Security Research Centre of Concordia University.

S. Ghaffarzadegan, M. Debbabi, and C. Assi are with Concordia University, Montréal, Canada (email: first.last@concordia.ca)

B. Nour and M. Pourzandi are with Ericsson Research, Ericsson, Montréal, Canada (email: first.last@ericsson.com).

mission planning. AUTOSIGMA automates a core bottleneck in this loop by transforming narrative threat intelligence into structured Sigma rules.

Although recent work has made progress toward automating certain CTI processing tasks, there is still no end-to-end solution capable of taking raw narrative reports and producing validated Sigma rules, aligned with MITRE ATT&CK and applicable to different attack types, whether living-off-the-land, cloud-based, or otherwise. To address this gap, we present AUTOSIGMA, an end-to-end solution that transforms unstructured CTI reports into precise, deployable Sigma rules. Our contributions can be summarized as follows:

- *Fully automated pipeline:* Unlike prior work [10], [11], AUTOSIGMA performs contextual analysis, template-driven rule retrieval, and generative refinement in a unified and automated pipeline that helps in reducing manual effort.
- *Attack scenario enrichment:* AUTOSIGMA enhances attack understanding by using external cybersecurity knowledge bases to fill in missing details and extend raw threat descriptions, which enables robust rule generation even with sparse input.
- *Attack decomposition:* AUTOSIGMA decomposes complex attack scenarios into smaller, atomic steps aligned with known detection logic. This improves the accuracy, granularity, and modularity of the generated Sigma rules.
- *Template-based rule grounding:* By retrieving similar Sigma rules from validated repositories, AUTOSIGMA uses them as templates to guide and constrain rule generation, which ensures syntactic correctness and contextual alignment with industry best practices.
- *LLM-as-a-Judge refinement:* AUTOSIGMA incorporates a dual-LLM feedback mechanism that iteratively validates and improves candidate rules, which helps in mitigating hallucination, enhancing quality, and ensuring rule validity.
- *Real-world validation:* We empirically evaluate AUTOSIGMA on multiple public security blogs and also APT reports (APT41, APT28, and APT29) against state-of-the-art solution and LLM models, demonstrating high-fidelity rule generation, accurate threat behavior capture, and readiness for deployment in operational SIEM environments.

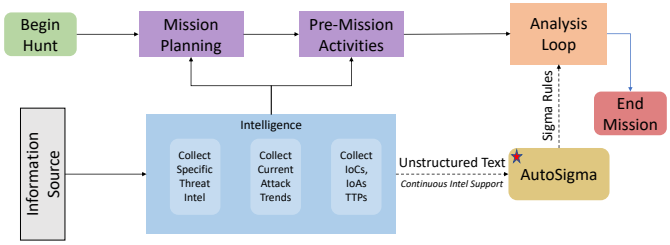


Fig. 1: Threat hunting process [9] with emphasis on our work focus (★).

Advantages of AUTOSIGMA: With its LLM-driven refinement mechanism, AUTOSIGMA offers several advantages over prior approaches.

First, unlike LLMCLOUDHUNTER [10] that requires highly detailed reports, AUTOSIGMA can compensate for incomplete intelligence by enriching attack descriptions with external knowledge bases, and thus accelerate rule generation and provide more threat coverage.

Second, unlike traditional single-pass generation or standard self-refinement methods [10], AUTOSIGMA introduces a feedback refinement loop driven by a dual LLM-as-a-Judge that aims to generate rules and refine them for better accuracy. This approach ensures consistent formatting and syntax of Sigma rules, which reduces human errors and improves detection accuracy.

Finally, AUTOSIGMA eases the operationalization of threat intelligence to be used in security operations without requiring expert rule engineers for every new threat report.

II. MOTIVATION AND PROBLEM STATEMENT

A. Motivation

Sigma rules are a widely adopted, vendor-agnostic signature format (written in YAML) for encoding CTI as detection logic across diverse SIEM platforms [3]. This portability makes Sigma particularly valuable, but also makes relevant rule creation even more critical. In practice, security teams often need to write new Sigma rules to support proactive threat hunting and to address diverse operational contexts. Traditional methods for manually crafting Sigma rules are not only time-consuming and error-prone, but also require significant domain expertise, limiting their scalability and responsiveness to emerging threats. This manual workflow cannot scale and urgently calls for automation [12]. Despite the existence of open-source Sigma rule repositories, these repositories cannot keep pace with the rapid evolution of adversarial tactics.

CTI reports (*e.g.,* Fig. 2) are inherently free-form and heterogeneous, often blending detailed attack steps with benign observations and inconsistent formatting. Automated parsing of such reports into Sigma's structured conditions is extremely challenging. Prior research has proposed various NLP-driven extraction techniques, but these generally yield only limited structured outputs (such as identification of IoCs, or MITRE ATT&CK techniques) that still require extensive manual interpretation [4], [13]. Moreover, even the existing Sigma rule corpus is far from complete: study [10] found that only 60 of 193 MITRE ATT&CK patterns had a corresponding Sigma rule (approximately 31% coverage). In other words, many behaviors documented in CTI may have no ready-made detection rule at all, forcing analysts to fill in gaps manually.

> Starting on January 20, 2020, APT41 used the IP address 66.42.98[.]220 to attempt exploits of Citrix Application Delivery Controller (ADC) and Citrix Gateway devices with CVE-2019-19781 (published December 17, 2019).
> The initial CVE-2019-19781 exploitation activity on January 20 and January 21, 2020, involved execution of the command 'file /bin/pwd', which may have achieved two objectives for APT41. First, it would confirm whether the system was vulnerable and the mitigation wasn't applied. Second, it may return architecture-related information that would be required knowledge for APT41 to successfully deploy a backdoor in a follow-up step.

Fig. 2: Excerpt from Mandiant's APT41 threat report [14]. This example illustrates the unstructured nature and narrative style of CTI reports, which pose challenges for automated rule extraction.

B. Problem Statement

Most methods [10], [11], [22] rely heavily on the quality and structure of the input report, which means they produce inconsistent results when faced with unstructured or poorly formatted documents.

TABLE I: Comparison table of knowledge extraction solutions from CTI.

Reference	Year	Technique	Dataset	Target Env.	Output Type	Output			Augmentation		Extraction		
						Act.	Rob.	WF	KBs	Temp.	Rel.	IoCs	TTPs
NLP-based Solutions													
TTPDrill [4]	2017	Unsupervised NLP	Symantec	On-premise	STIX	✗	✗	✗	✗	✗	✓	✓	✓
ThreatRaptor [5]	2021	Unsupervised NLP	DARPA TC	On-premise	Behavior Graph + SQL	✗	✗	✗	✗	✗	✓	✓	✓
BERT-based and BiLSTM Solutions													
CASIE [15]	2020	BiLSTM	CyberWire	On-premise	Knowledge Graph	✗	✗	✗	✗	✗	✓	✓	✗
Extractor [13]	2021	BERT-BiLSTM	Mixed Sources	On-premise	Behavior Graph	✗	✗	✗	✗	✗	✓	✓	✓
Open-CyKG [16]	2021	BiLSTM	MalwareDB	On-premise	Knowledge Graph	✗	✗	✗	✗	✗	✓	✗	✓
SecIE [17]	2022	BERT	CVE	On-premise	Knowledge Graph	✗	✗	✗	✓	✗	✓	✓	✗
TriCTI [18]	2022	BERT	Custom	On-premise	Labeled IoCs	✗	✗	✗	✗	✗	✗	✓	✗
LADDER [6]	2023	BERT	Custom	On-premise	Knowledge Graph	✗	✗	✗	✓	✗	✓	✓	✓
CyberEntRel [19]	2024	RoBERTa-BiGRU-CRF	Custom	On-premise	Knowledge Graph	✗	✗	✗	✓	✗	✓	✓	✗
FlowGuardian [20]	2025	BERT	Custom	On-premise	TestFlow	✗	✗	✗	✗	✗	✓	✓	✓
LLM-based Solutions													
Purba, Chu [21]	2023	GPT-3.5	Twitter Posts	On-premise	Labeled IoCs	✗	✗	✗	✗	✗	✗	✓	✗
acTion [22]	2023	GPT-3.5	Custom	On-premise	STIX	✗	✗	✗	✓	✗	✓	✓	✓
Liu *et al.* [23]	2023	ChatGPT	Custom	On-premise	Knowledge Graph	✗	✗	✗	✓	✗	✓	✓	✓
LLM-TIKG [24]	2023	LLaMA-2-7B (fine-tuned)	Custom	On-premise	Knowledge Graph	✗	✗	✗	✓	✗	✓	✓	✓
Fengrui *et al.* [25]	2024	LLaMA-2-7B (fine-tuned)	ATT&CK STIX Data	Hybrid	MITRE ATT&CK TTPs	✗	✗	✗	✓	✗	✓	✓	✓
IntelEX [11]	2024	LLM	APT CTI	On-premise	Knowledge Graph + Sigma Rule	✓	✗	✗	✓	✗	✓	✓	✓
LLMCloudHunter [10]	2025	GPT-4o	Custom	Cloud	Sigma Rules	✓	✗	✗	✗	✗	✓	✓	✓
AutoSigma	2025	LLM (LLM-as-a-Judge)	Custom	Cloud & On-premise	Flow of Sigma Rules	✓	✓	✓	✓	✓	✓	✓	✓

Act.: *Actionable*, **Rob.**: *Robust to input quality*, **WF**: *Workflow*, **KBs**: *External Knowledge Base*, **Temp.**: *Template Retrieval*, **Rel.**: *Relations*.

Other approaches [6], [24] lack contextual enrichment; they do not incorporate additional knowledge sources to augment the information in the report, therefore limiting the accuracy and relevance of the generated rules.

Moreover, the insights extracted by these systems often lack the granularity and structured format necessary for direct integration into SIEM environments. Consequently, security teams must manually refine or complete these partially generated rules, undermining the goal of full automation.

More fundamentally, there is a conspicuous gap in research when it comes to translating narrative threat intelligence into actionable content. Only a few studies [10], [11] have attempted to bridge this gap, and they face several obstacles: (i) the inherent complexity of converting natural-language descriptions of attacks into the formal structure of Sigma rules, (ii) the difficulty of correlating the sequence of extracted attack steps with existing detection rule patterns, which is essential to ensure that any generated rule is consistent with known attack behaviors and truly relevant.

These challenges have so far prevented the development of a solution capable of producing high-quality, ready-to-use Sigma rules from unstructured threat reports.

III. Related Work

CTI plays a critical role in cybersecurity by providing timely insights into threat actors' tactics and techniques [12]. In this section, we review existing solutions for CTI knowledge extraction and Sigma rule generation. Table I compares AutoSigma and other tools for knowledge extraction from CTI reports.

CTI Knowledge Extraction: Early methods for knowledge extraction from CTI include TTPDrill [4], which uses unsupervised NLP to extract threat actions from Symantec reports and map them to MITRE ATT&CK techniques. EXTRACTOR [13] advances this by using BERT-BiLSTM to convert unstructured CTI into formal evidence graphs for threat hunting. ThreatRaptor [5] combines action and indicator extraction into structured behavior graphs. CASIE [15] and Open-CyKG [16] use BiLSTM-based models to extract entities and relations from reports to populate threat knowledge graphs. These works structure threat knowledge but do not produce detection rules or offer mechanisms to assess correctness. Recently, LLMs have been applied to improve knowledge extraction. LADDER [6] and SHIELD [26] demonstrate entity extraction and threat reasoning using contextual modeling, with SHIELD employing iterative refinement to improve reliability.

Sigma Rule Generation:

LLMCloudHunter [10] uses GPT-4 to parse cloud-focused CTI reports and generate candidate Sigma rules. It shows high success in rule compilation and precision for cloud environments, but lacks a rule validation mechanism, making its outputs vulnerable to hallucinations. Furthermore, its design depends heavily on input quality and assumes well-parsed paragraphs, limiting robustness to noisy or complex CTI. Other systems, such as SHIELD [26] and acTion [22], apply LLMs for alert explanation or constructing technique knowledge graphs from structured threat sources. While relevant for enhancing threat visibility, they do not generate deployable Sigma detection rules.

Similarly, recent works [21], [23]–[25] apply LLMs for IoC classification or knowledge base population but lack the full pipeline required for rule synthesis. IntelEX [11] is a notable system that leverages in-context prompting with external knowledge bases and a single-pass LLM-as-a-Judge mechanism to extract structured multi-step threat intelligence. It achieves strong performance on TTP and IoC extraction and shows early potential for generating Sigma rule candidates. However, IntelEX remains limited to content present in the report itself and does not perform template-based rule enrichment or similarity retrieval, which constrains rule generalizability and robustness.

In contrast, AutoSigma introduces an end-to-end solution for CTI to Sigma translation. It performs contextual analysis and graph-based similarity matching to retrieve the most relevant Sigma rule templates, enriches them with external intelligence, and completes the rule using generative LLMs. A dual-LLM validation process (LLM-as-a-Judge) iteratively ensures syntactic correctness and semantic alignment with the

CTI content. Unlike LLMCLOUDHUNTER, which performs one-pass generation on curated input, AUTOSIGMA is robust to formatting variations and enhances rule fidelity via knowledge-driven retrieval and multi-step verification. Unlike INTELEX, which remains constrained to within-report content, AUTOSIGMA goes beyond the report to build a complete and context-rich attack scenario grounded in known detection patterns.

IV. THREAT MODEL AND ASSUMPTIONS

In-scope threats: Our solution targets sophisticated, multi-stage cyberattacks, such as APTs, that are described in unstructured documents (*e.g.,* threat reports, incident response documentation). These attacks typically exploit software vulnerabilities (*e.g.,* Common Vulnerabilities and Exposures – CVE), leverage known adversary tactics and techniques, and proceed through a kill-chain involving initial access, lateral movement, and impact.

Out-scope threats: AUTOSIGMA does not aim to detect physical security incidents, insider threats not documented in reports, or attacks that do not leave observable artifacts suitable for Sigma rule generation. Similarly, zero-day threats with no previous contextual documentation are not within the immediate scope of detection unless indirectly referenced in CTI.

System Assumptions: We assume that the input is an unstructured threat report that describes one or more attack scenarios. Although our experiments focus on APT reports, our work is not limited to them; the input can be any textual description of malicious activity, including incident response reports, blog posts, or informal threat notes. Our methodology does not require the input text to be cleanly structured or annotated. We assume input reports are authentic and not intentionally manipulated.

We assume that each report may contain multiple discrete attack steps. As such, the system is expected to generate not one, but a sequence of Sigma rules, each corresponding to a specific attack action or tactic. These rules may vary in granularity depending on the level of detail available in the input. Finally, we assume the correctness of the external knowledge bases that the solution is leveraging.

V. AUTOMATED SIGMA RULES GENERATION

A. System Overview

1) Overview:

AUTOSIGMA, depicted in Fig. 3, is designed to automate the transformation of APT reports into high-quality, actionable Sigma rules. The architecture follows a modular pipeline approach, consisting of four main components.

The first component of AUTOSIGMA is Contextual Analysis (Section V-B), where it performs text preprocessing and applies Named Entity Recognition (NER) and TTP extraction to identify key elements within the unstructured threat report. Next, in the Attack Understanding component (Section V-C), the extracted entities and tactics are analyzed to infer possible attack scenarios. These scenarios are then enriched using external knowledge bases such as MITRE ATT&CK and NVD, allowing AUTOSIGMA to incorporate intelligence beyond what is explicitly stated in the report, thereby mitigating the limitations imposed by poor-quality or incomplete input. Subsequently, complex attacks are decomposed into atomic attack steps, enabling precise reconstruction of the attack chain and ensuring that generated rules are both comprehensive and granular. Following this, the Template Matching component (Section V-D) employs a semantic similarity search over a curated repository of validated Sigma rules. By embedding both the extracted attack step and the rule corpus into a shared vector space, AUTOSIGMA retrieves the closest matching rule template, grounding its output in previously vetted detection logic. This retrieval-based grounding is a novel contribution that ensures generated rules inherit the structural correctness and semantic intent of battle-tested Sigma patterns, something not addressed in prior approaches. Finally, in the Rule Generation component (Section V-E), AUTOSIGMA uses an LLM-as-a-Judge paradigm to iteratively refine and validate each Sigma rule. A generation model proposes a candidate rule, while a validator model critiques and suggests corrections. This adversarial loop is repeated until the output satisfies both syntactic validity and semantic alignment with the source scenario. Through this pipeline, AUTOSIGMA combines multi-stage contextual reasoning, external enrichment, template-driven rule retrieval and iterative validation to produce robust Sigma rules that are both immediately deployable and semantically faithful to the underlying threat report.

2) Novelty:

AUTOSIGMA introduces multiple novel elements that distinguish it from prior approaches. First, it performs end-to-end automation of the Sigma rule generation process, starting from raw unstructured threat reports to producing relevant, deployable rules. Second, it integrates external knowledge sources such as the MITRE ATT&CK knowledge base and the National Vulnerability Database (NVD)[1] to enrich attack context and fill in gaps from incomplete reports. Third, it constructs the entire attack chain by decomposing each scenario into a sequence of attack steps, each mapped to a Sigma rule. Fourth, it leverages an up-to-date repository of official Sigma rules to retrieve and adapt existing templates through similarity search. Finally, a dual-LLM feedback loop, where a rule generator and a rule validator iteratively collaborate to refine the quality of the generated Sigma rules.

3) Design Challenges:

Designing AUTOSIGMA required addressing several challenges. CTI reports are typically written in free form and lack consistent structure, making it difficult to directly extract entities or behavior sequences. The availability of high-quality, labeled cybersecurity data is limited, which constrains the use of traditional supervised models for tasks like NER. Additionally, large language models, while powerful, are prone to hallucinations or producing outputs that are syntactically incorrect or semantically misaligned with the input. Controlling these errors and handling failure cases gracefully was essential to ensuring pipeline reliability. AUTOSIGMA addresses these challenges through contextual analysis, leveraging external knowledge bases, and using LLMs for tasks such as NER

[1]NVD: https://nvd.nist.gov/

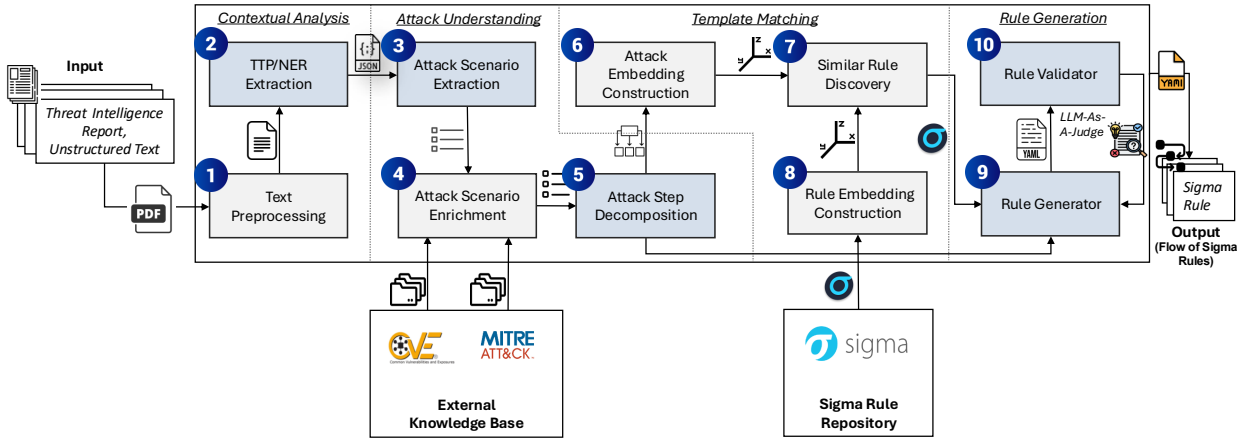


Fig. 3: High-level overview of AUTOSIGMA and its components (components shown by blue are based on LLM).

extraction. Additionally, the LLM-as-a-Judge refinement loop improves the quality of the final results.

B. Contextual Analysis

Definition 1. Let $\mathcal{D}$ represent the set of unstructured CTI documents. We define $\mathcal{S}$ as the set of preprocessed text segments (derived via Component 1 in Fig. 3) and $\mathcal{X}$ as the multidimensional set of extracted cybersecurity entities, including NERs, IoCs, and MITRE ATT&CK TTPs (derived via Component 2). The resulting contextual representation space is defined as $\mathcal{C} \subseteq 2^{\mathcal{S}} \times 2^{\mathcal{X}}$. The contextual analysis stage is modeled as a mapping function $F_{\text{ctx}} : \mathcal{D} \to \mathcal{C}$, such that for any report $d \in \mathcal{D}$, the function yields a structured tuple $c_d = (S_d, X_d)$, where S_d captures the preprocessed narrative and X_d isolates the contextual information.

The first component of AUTOSIGMA is responsible for extracting meaningful threat intelligence from raw threat reports. Since these reports vary widely in structure and format, this module applies preprocessing techniques to ensure consistency, remove irrelevant content, and isolate the information most pertinent to subsequent analysis.

1) Text Preprocessing:

AUTOSIGMA ingests threat reports and performs rigorous text preprocessing, which prepares the data for further analysis. This phase involves several NLP techniques [27] to refine raw, unstructured data:

(a) extracting the text given a threat report; (b) text cleaning, which removes the irrelevant data from the text and prepares the text for further analysis; (c) normalization, which standardizes the data for consistency and improved algorithm performance; and (d) segmentation, which allows us to feed the language models with input in manageable pieces, optimizing the performance of the LLMs in our architecture by providing context segment-by-segment instead of in one large block [28].

2) TTP and NER Extraction:

Various NLP solutions have been proposed for extracting named entities and mapping tactics, techniques in cybersecurity contexts, such as PELAT [29], SMET [30], and Adema [31]. However, these approaches often rely on annotated datasets and may lack adaptability to diverse, unstructured threat reports. To address these limitations, AUTOSIGMA uses LLM for NER extraction, leveraging its capabilities to generalize from limited data and understand complex contexts. We also experimented with a BERT-based NER model [32], but the LLM approach yielded better results given the limited training data available for these entity types. Once the key entities are extracted from the report, AUTOSIGMA analyzes them to find tactics, techniques, and aligns the information with the MITRE ATT&CK knowledge base. This alignment provides a precise categorization of the described attacker behaviors, ensuring that subsequent processing steps have a structured understanding of the threat context.

Example 1. Consider the following excerpt from a threat report:

"APT41 leveraged CVE-2019-19781 to deploy Cobalt Strike beacons, facilitating lateral movement across the network."

In this case, AUTOSIGMA identifies APT41 as the threat actor, CVE-2019-19781 as the exploited vulnerability, and Cobalt Strike as the malware used in the operation. Based on this, it infers the corresponding MITRE ATT&CK techniques: Exploit Public-Facing Application (T1190), Command and Control through Beaconing (T1071.001), and Lateral Movement via Remote Services (T1021).

C. Attack Understanding

Definition 2. Let $\mathcal{A}$ represent the set of enriched and decomposed attack steps derived from Components 3 , 4 , and 5 . Each element $a \in \mathcal{A}$ describes a single malicious behavior together with its associated indicators, MITRE ATT&CK annotations, and enriched information (*e.g.,* CVE metadata). The attack understanding phase is modeled as a transformation $f_{\text{au}} : \mathcal{C} \to 2^{\mathcal{A}}$, which maps a contextual representation $c_d \in \mathcal{C}$ to a discrete, ordered set of enriched and decomposed attack steps $A_d = \{a_1, a_2, \ldots, a_n\} \subseteq \mathcal{A}$.

While NER extraction and MITRE ATT&CK technique mapping provide a foundational understanding of threat reports, they are often insufficient for generating high-quality, specific Sigma rules. This is because the raw extracted entities alone may lack context, depth, or completeness, especially in cases where the report is vague or incomplete. Therefore, AU-

toSIGMA performs an additional layer of analysis to construct a more coherent and actionable representation of the attack. In this phase, AUTOSIGMA goes beyond simple extraction, meaning it not only identifies entities and techniques, but also enriches them using external cybersecurity sources. For example, if a CVE is mentioned, AUTOSIGMA queries the National Vulnerability Database (NVD)[2] to retrieve its detailed description, severity, and affected software. Similarly, if a known threat actor is referenced, information from the MITRE ATT&CK knowledge base is used to retrieve related tactics, tools, and historical campaigns. Furthermore, to ensure that the Sigma rules generated are both granular and actionable, this component decomposes complex attack descriptions into a sequence of atomic attack steps. Each step corresponds to a specific malicious behavior or tactic, and ultimately maps to a separate Sigma rule. This decomposition is critical for generating accurate and modular detection logic.

1) Attack Scenario Extraction:

Using the identified NER entities and TTPs, the system reconstructs meaningful attack scenarios mentioned in the report. This step synthesizes fragmented information into structured attack scenarios. For example, APT41 exploited CVE-2019-19781 to gain initial access to the target system by utilizing FTP to download a malicious payload.

These attack scenarios provide structured context for further enrichment and Sigma rule generation. By the end of this step, AUTOSIGMA produces a structured representation of the extracted intelligence, making it suitable for subsequent enrichment and detection rule generation.

2) Attack Enrichment:

To provide additional context to attack scenarios, AUTOSIGMA queries multiple cybersecurity knowledge bases. The enrichment process involves, but is not limited to, the following:

- *CVE Enrichment:* If an attack uses a CVE (*e.g.,* CVE-2019-19781), AUTOSIGMA retrieves relevant details such as the description of the vulnerability, CVSS score and severity rating, affected software, and versions. This information is sourced from external knowledge bases. In this work, we used the CVE repository as our knowledge base. The repository contains over 277,000 CVE records, each providing standardized information about publicly known cybersecurity vulnerabilities, including descriptions, severity scores, and affected products [33].
- *Threat Actor Enrichment:* We also use MITRE ATT&CK as a knowledge base, where AUTOSIGMA queries it to extract additional information related to the attack. This information can be captured in different components, such as tools, malware, and software that a threat actor might use.

3) Attack Step Decomposition:

Given that an attack scenario often contains multiple attack behaviors or subgoals, it is not feasible to generate a single Sigma rule that effectively captures all of them. Therefore, AUTOSIGMA decomposes each enriched scenario into smaller, discrete attack steps. This decomposition plays a critical role in aligning each step with existing detection logic more effectively. Unlike prior approaches [10], [11] that attempt to encapsulate an entire attack scenario within a single rule, our method breaks down complex threats into distinct, manageable components, enabling the generation of multiple, focused flows of Sigma rules. This not only enhances the precision of each rule but also promotes modularity and scalability in detection coverage. Each step isolates a specific tactic or action taken by the adversary, which in the next phase will be mapped to a detection rule. To perform this decomposition, AUTOSIGMA uses a prompting-based approach with an LLM, where the enriched attack description is input and the model is guided to split the scenario into atomic substeps. This step ensures that every component of an attack is isolated and can be individually analyzed for detection logic, thereby eliminating ambiguity and improving the fidelity of the final detection rules. An example of the output for this step is illustrated in Appendix A. The prompting strategy is also discussed in Appendix B.

[2]NVD: https://nvd.nist.gov/

D. Template Matching

Definition 3. Let $\mathcal{R}$ denote the set of all Sigma rules within the SigmaHQ repository. The template matching phase is defined as a retrieval function $f_{\text{tm}} : \mathcal{A} \to 2^{\mathcal{R}}$. For any enriched attack step $a \in \mathcal{A}$, the function yields a non-empty, finite subset $T_a \subseteq \mathcal{R}$, representing the candidate templates that exhibit the highest semantic similarity to a within the shared vector space $\mathcal{V}$ (derived via Components 6 , 7 , and 8). This is expressed as $T_a = \{r \in \mathcal{R} \mid \text{sim}(a, r) > \tau\}$, where sim is the BERTScore metric and τ is a relevance threshold.

To improve the relevance and quality of the generated rules, we introduce a novel step: retrieving similar existing Sigma rules to be used as templates for generating new ones. In the third stage, AUTOSIGMA prepares to leverage existing knowledge of known detection rules by embedding both the attack steps derived from the report and a large collection of known and valid Sigma rules into a common vector space. By representing attack descriptions extracted from the attack understanding phase, and Sigma rules as numerical embeddings, AUTOSIGMA can perform efficient similarity searches to identify which known rules are most relevant to the attack steps at hand. This component bridges the gap between the enriched attack intelligence and the rule generation phase by providing a starting template for each detection rule AUTOSIGMA needs to generate.

1) Attack Embedding Construction:

Each attack description produced in the previous stage is converted into a vector embedding using a BERT-based model [32]. This transformation captures the semantic meaning of the attack description in a high-dimensional space. The motivation behind this step is to enable comparison with existing Sigma rule embeddings, by representing both the attack descriptions and Sigma rules in the same vector space.

2) Rule Embedding Construction:

AUTOSIGMA uses an up-to-date collection of Sigma rules, drawn from the latest SigmaHQ[3] repository. To organize this repository for efficient search, we built a graph-based representation of those Sigma rules. This representation acts as an indexing method that accelerates rule retrieval during the next stage of AUTOSIGMA. In SigmaHQ repository, rules are already arranged hierarchically into categories such as *application*, *cloud*, *linux*, and *macos*, each containing multiple subcategories including *linux* → *auditd*, *file_event*, *process_creation* and *cloud* → *aws*, *azure*, *gcp*, *m365*. We convert this directory structure into a lightweight graph in which top-level categories form the first layer, subcategories form the second layer, and individual Sigma rules form the third layer. This abstraction enables efficient traversal and indexing of the rule set during the next stage. We formally define this structure as a directed acyclic graph (DAG) G = (V, E), where nodes represent the repository root, platforms, categories, and individual Sigma rules, and edges encode hierarchical relationships between them. The root node corresponds to the SigmaHQ repository, followed by platform nodes (e.g., linux, windows, cloud), category nodes (e.g., process_creation, file_event), and finally rule nodes. During retrieval, given an attack step, AUTOSIGMA first narrows the search space by selecting relevant platform nodes, then traverses to their corresponding categories, and finally evaluates candidate rules within those categories using semantic similarity. This hierarchical traversal reduces the search space and improves retrieval efficiency. Because the SigmaHQ repository is continuously updated, AUTOSIGMA automatically fetches the most recent set of Sigma rules whenever updates are available. With each execution of the solution, AUTOSIGMA refreshes the SigmaHQ repository to ensure that the most updated rules are incorporated into the pipeline. It then incorporates these into its knowledge base, ensuring that even cutting-edge detection techniques are considered during rule generation. Deprecated rules are pruned and new rules are indexed without manual intervention, enabling AUTOSIGMA to build on top of the recent rule set. We leverage this repository to find existing Sigma rules that closely match the attack step in question. If a relevant rule is found, we refine and build upon it. Otherwise, we use the rules as one-shot learning examples for the LLM.

3) Similar Rule Discovery:

The repository of Sigma rules serves as the foundation for the similarity matching process. With every rule indexed and embedded, AUTOSIGMA can perform rapid searches to find which existing rules are most relevant to a given attack step. Moreover, because the repository is kept current, the framework ensures that it is comparing against the latest known detection patterns. This means the output rules will align closely with community-curated detection techniques and will not reinvent rules that already exist in the repository. In order to achieve this, we compute BERTScore [34] between an attack step embedding and Sigma rule embeddings to measure how closely a given attack description matches the scope of any known detection rule. The reason why we use BERTScore instead of other similarity scores, such as cosine similarity or BLEU [35], is that this method allows us to capture not only exact word matches but also the overall meaning, fluency, and order of the output, making it more effective than traditional n-gram-based metrics like BLEU. This embedding-based approach allows the system to efficiently scan a large corpus of rules and pinpoint those that are contextually most similar to the new attack scenario.

[3]SigmaHQ: https://github.com/SigmaHQ/sigma

E. Rule Generation

Definition 4. Let σ denote the set of syntactically valid Sigma rules. The rule generation component (Component 9) and its iterative validation loop (Component 10) are modeled as a refinement function $f_{\text{rg}} : \{(a, T_a) \mid a \in \mathcal{A}, T_a \subseteq \mathcal{R}\} \to 2^{\sigma}$. For a given enriched attack step a and its associated template set T_a, the function outputs a set of finalized rules $R_a \subseteq \sigma$. A rule r is included in R_a only if it satisfies the predicate $\text{valid}(r) \wedge \text{aligned}(r, a)$, where these conditions are validated through the LLM-as-a-Judge refinement loop.

In the final phase, AUTOSIGMA takes the candidate Sigma rule templates identified in the previous stage and refines them into high-fidelity detection rules tailored to the specific attack scenarios.

1) Rule Generator:

The rule generation component is designed to transform enriched attack steps into high-quality, executable Sigma rules. The primary goal of this stage is to take the best-matching Sigma templates (see Section V-D) and enrich them for the given attack description while ensuring the resulting rule meets several key criteria. It must accurately reflect the attack techniques described in the source reports and cover all relevant IoCs along with the associated adversary behaviors. At the same time, the rule should maintain strong detection capabilities with minimal false positives, and it must strictly adhere to the Sigma rule format for seamless integration with SIEM platforms. To fulfill these objectives, AUTOSIGMA integrates an LLM-as-a-Judge mechanism [36] in which two LLMs work in an adversarial collaboration to iteratively refine the Sigma rules being generated. Details are provided in Appendix B.

2) LLM-as-a-Judge:

The rule generation phase consists of two distinct LLMs, they can be the same underlying LLMs or they can be two different LLMs, interacting with each other:

1. Rule Generator (LLM-1): This first LLM (the "rule generator") takes the retrieved Sigma rule template along with the context of the attack step and generates an initial Sigma rule tailored to the scenario. Essentially, LLM-1 fills in or modifies the template's fields (such as the rule title, description, detection section filters, and tags) using the information from the attack step. The output at this stage is a draft rule hypothesized to detect the described attack.

2. Rule Validator (LLM-2): The second LLM acts as a validator or *judge*. It examines the proposed Sigma rule from LLM-1 and evaluates it against a set of predefined quality checks:

- Does the rule accurately detect the described attack technique and behaviors?

- Is the detection logic valid, sensible, and free of obvious gaps or mistakes (*e.g.,* correct field names, appropriate use of conditions)?
- Is the rule properly formatted according to YAML and Sigma specifications?

Based on this evaluation, LLM-2 provides a quantitative score or verdict on the rule's quality and generates structured feedback highlighting what needs improvement. For example, the validator might point out that a certain condition is too broad (risking false positives) or that a required field is missing or misformatted. The interplay of these two models, one generating and the other validating, forms an adversarial learning loop. LLM-1 and LLM-2 do not operate in isolation but rather communicate through the intermediate Sigma rule drafts and feedback.

Iterative Refinement Process: Using the generator-validator setup above, AUTOSIGMA iteratively refines each Sigma rule through multiple rounds, as follows:

1) **Initial Draft:** LLM-1 generates an initial enriched rule based on the attack description and the selected template (as described in Section V-E1).
2) **Validation and Scoring:** LLM-2 evaluates this draft, checking its accuracy, logic, and format, and returns a score along with feedback on how to improve the rule.
3) **Feedback-Driven Revision:** LLM-1 incorporates LLM-2's feedback and revises the Sigma rule, for example, by tightening detection conditions, adding missing IoCs, or correcting format errors.
4) **Re-Validation:** The updated rule is sent back to LLM-2 for another round of evaluation.
5) **Repeat Until Optimal:** Steps 2-4 repeat, with the rule steadily improving each time, until the validator LLM judges that the rule meets the desired quality threshold. In practice, this loop converges after a few iterations, resulting in a rule that LLM-2 can no longer fault on the predefined criteria.

This refinement loop continues until the Sigma rule is both syntactically valid and semantically robust for detection. One important note is that the LLM-as-a-Judge mechanism is not designed to handle hallucinations of the LLMs; rather, its purpose is to improve the quality of the generated output. Hallucination control is instead addressed through the use of lower temperature settings and fine-tuned LLMs (see Section VII, RQ3.3). Additionally, we observed in very few cases (1-2 instances) that the refinement loop could get stuck in an infinite loop. To handle this, we introduced a maximum iteration limit of 5 in the pipeline.

VI. IMPLEMENTATION

We build a Proof of Concept (PoC)[4] using Python 3 programming language. Our experiments were conducted on Intel Xeon E312xx, 6 vCPU @ 2.69Ghz, with 32GB RAM running Ubuntu 20.04. Further details regarding the PoC are provided in Appendix C.

[4]AUTOSIGMA's Demo: https://youtu.be/iSr6IurQ6BM

A. LLM Models

There are generally two methods to work with LLMs: (i) using a cloud-based LLM, trained and deployed by a third-party, or (ii) using a locally hosted LLM. Each approach has its own advantages and disadvantages.

Cloud-based LLM access does not require extensive computational resources and simplifies request handling; however, it is often associated with usage costs, and concerns remain about data confidentiality and the possible influence of concurrent API requests on output quality. Running LLMs locally ensures data privacy and eliminates risks associated with multi-tenant environments, but demands significant computing resources. Without sufficient hardware, one must use lighter LLM variants, which may negatively impact performance. For our evaluation, we used both methods:

- *Local-based LLM:* We employed *Lily-Cybersecurity-7B-v0.2*[5], a fine-tuned variant of Mistral-7B-Instruct-v0.2, specifically trained for cybersecurity tasks. The training encompassed a broad spectrum of cybersecurity domains, including but not limited to APT management, malware analysis, incident response, and secure software development lifecycle. We configured the model with a temperature of 0.1 to minimize randomness and a maximum token length of 512 to ensure concise and relevant outputs.
- *Cloud-based LLM:* We used *ChatGPT-4o-mini*[6] and *Llama-3.3-70b*[7] through OpenAI and Llama's APIs. We also set the temperature parameter to 0.1 and the maximum token length to 512 to maintain consistency in output quality and length.

More details about the LLMs are provided in Appendix E.

B. Datasets

Threat Reports: We use threat reports from the APTNotes repository[8], which gathers different threat intelligence from different vendors (*e.g.,* Mandiant, CrowdStrike). Given the lack of a comprehensive dataset, we focused only on APT41, APT28, and APT29, yet the solution can work with any report. Appendix D provides a summary of the CTI reports used in our evaluation. To further evaluate different solutions, we also used the dataset released by LLMCLOUDHUNTER [10], comprising 20 cloud-security blogs (Azure/AWS/GCP) paired with their manually crafted Sigma rules.

Knowledge Base: AUTOSIGMA incorporates two external knowledge bases. The NVD, which provides a comprehensive repository of CVEs. We utilize this database to extract relevant information about identified vulnerabilities. The second is the official SigmaHQ repository, which contains more than 3,600 rules spanning over 60 detection domains. These rules are distributed across several major categories, including but not limited to, operating system domains (*e.g.,* Windows, Linux, macOS), cloud environments (*e.g.,* AWS, Azure, GCP, M365)

[5]Lily Cybersecurity Model: https://huggingface.co/segolilylabs/Lily-Cybersecurity-7B-v0.2

[6]ChatGPT 4o-mini: https://platform.openai.com/docs/models/gpt-4o-mini

[7]Llama-3.3-70B: https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct

[8]APTNotes repository: https://github.com/aptnotes/data/

and more. This diversity reflects the breadth of the Sigma rule ecosystem.

VII. Evaluation

We adopt three research questions that jointly assess AutoSigma quantitatively and qualitatively: (i) benchmarking against state-of-the-art solution; (ii) cross-APT evaluation against LLM models, and (iii) design-choice and robustness studies isolating the effects of the LLM-as-a-Judge loop and model variation and sampling temperature.

RQ1. How effective is AutoSigma in generating high-quality and behavior-aligned Sigma rules compared to a state-of-the-art solution?

We evaluate AutoSigma, using LLMCloudHunter dataset, in two configurations AutoSigma using cloud-based LLMs and AutoSigma using a Local LLM, and compare against LLMCloudHunter.

All systems are assessed with seven complementary metrics: (i) *No. Rules:* number of generated Sigma rules per report; (ii) *Rule Validity:* percentage of rules that are syntactically valid and convertible to SIEM queries (*e.g.,* Splunk/ELK) using sigma-cli[9]; (iii) *Condition Accuracy:* focuses on the correctness of the condition fields, which specify the relationship between various selection fields. (iv) *Rule Relevancy:* semantic similarity between each report and its generated rule set via BERTScore [37]. (v) *Partial Coverage:* number of generated rules that match at least one of the same MITRE ATT&CK techniques as a ground truth rule; (vi) *Full Coverage:* number of generated rules whose detection logic is equivalent to a ground truth rule; (vii) *Newly Discovered:* number of new rules generated by the model but not present in the ground truth; The results are summarized in Table II and discussed in the following sub-research questions.

TABLE II: Average results of AutoSigma against LLMCloudHunter using cloud-security blogs.

	AutoSigma (Cloud-based OpenAI)	AutoSigma (Local-based Lily)	LLMCloudHunter
No. Rules	21	25	8
Rule Validity	100	77.94	94
Condition Accuracy	100	100	100
Rule Relevancy	80.4	77.7	76.4
Partially Covered	97.40	84.41	80.9
Fully Covered	83	74.93	74
Newly Discovered	12	11	2

RQ1.1. Are the generated rules syntactically correct, semantically consistent with the input reports? Results for total number of rules and rule validity are shown in Table II. As we can see, AutoSigma in both cloud-based and local-based, outperformed the state-of-the-art solution. Due to the fact that LLMCloudHunter tends to generate the rules based on the telemetry information, such as IoCs mentioned in the report, this limits the overall number of rules produced. For rule validity, the cloud-based version of AutoSigma achieves a score of 100%, meaning every generated rule is syntactically valid and can be directly converted into a SIEM query. The local version, while producing a slightly lower validity percentage compared to LLMCloudHunter, primarily because it uses a lighter-weight LLM, still maintains a higher count of valid rules than LLMCloudHunter due to its larger output volume. This difference between the cloud-based and local-based configurations stems from their underlying LLM capabilities and generation precision, as further discussed in Appendix E. Furthermore, the condition accuracy metric shows that all generated rules, across all evaluated models, achieve 100% correctness in their condition fields. This indicates that the logical structure of each rule faithfully captures the intended detection behavior and can correctly operationalize the threat activity described in the reports.

RQ1.2. Do the generated rules correspond to behaviors explicitly described in the original reports? In this experiment, all models generate rules that remain semantically aligned with their corresponding CTI reports. However, as the rule relevancy (Table II) shows that both cloud-based and local-based versions of AutoSigma, achieve higher semantic similarity scores (80.4% and 77.7%, respectively) than the state-of-the-art solution (76.4%). This demonstrates AutoSigma's strength in maintaining contextual coherence throughout its Attack Scenario Extraction and Enrichment. Even with a higher number of generated rules, which typically increases the risk of semantic drift, AutoSigma preserves alignment with the original text due to its contextual analysis and LLM-as-a-Judge components, ensuring the final outputs remain faithful to the report's intent.

RQ1.3. To what extent do the rules generated capture the threat behaviors and MITRE ATT&CK techniques documented in the reports? As shown in Table II, both versions of AutoSigma stand ahead of LLMCloudHunter. In terms of partially covered rules, AutoSigma has covered 97.4% and 84.4%, versus 80.9% for LLMCloudHunter. For fully covered rules, AutoSigma has 83.0% compared to 74.0% obtained by LLMCloudHunter. This is due to the fact that LLMCloudHunter tends to focus on telemetry information such as IoCs when generating Sigma rules, which limits the overall number of rules produced. Moreover, LLMCloudHunter is specifically designed to filter cloud-based threat scenarios and therefore only generates cloud-based rules. Overall, all models perform reasonably well since they rely on the MITRE ATT&CK technique extraction at some stage of their process. However, AutoSigma demonstrates superior coverage by 8%, because its design does not have any domain constraint filters. Unlike LLMCloudHunter, AutoSigma identifies techniques across diverse contexts, resulting in broader and more comprehensive coverage.

RQ1.4. Can we identify new or previously uncovered Sigma rules? The newly discovered metric highlights the capacity of a model to uncover attack behaviors not explicitly reflected in the ground truth. Manual rule creation often overlooks such subtle variations or decomposed steps. Importantly, these newly discovered rules are not arbitrary, but correspond to attack behaviors described in the CTI reports that are not captured by other baselines. As such,

[9]SigmaCLI: https://github.com/SigmaHQ/sigma-cli

they contribute additional detection coverage by identifying relevant attacks that would otherwise remain undetected. As shown in Table II, AUTOSIGMA achieves an average newly discovered rule of 12 compared to LLMCLOUDHUNTER of 2, as AUTOSIGMA employs attack scenario extraction and decomposition components, which break complex attack narratives into simpler attacks that are more effectively expressed as Sigma rules. AUTOSIGMA not only focuses on telemetry information described in the threat report, but also leverages external knowledge bases. This process enables AUTOSIGMA to discover a greater number of novel rules (12 and 11 on average) compared to LLMCLOUDHUNTER.

RQ2. How does AUTOSIGMA perform relative to LLM baseline models in generating Sigma rules for APT reports?

To evaluate the performance of AUTOSIGMA against LLM models, we defined four metrics: (i) *Coverage:* evaluate the content retention from the report at two levels: (a) *IoC coverage* by checking if IoCs mentioned in the report have been covered by the generated rules, and (b) MITRE *coverage* by checking the covered techniques from the report and the generated rules. For this experiment we used Threat2Mitre [38] as our baseline; (ii) *Relevancy:* Semantic alignment between each report and its generated rule set using BERTScore embeddings; (iii) *Validity:* rules that are syntactically valid and convertible to SIEM queries (*e.g.,* Splunk/ELK) using sigma-cli; and (iv) *Adaptability:* Pairwise BERTScore similarities among *inputs* (reports) and among *outputs* (rules). It is important to mention that LLMCLOUDHUNTER is designed for cloud-specific content and does not produce any rules if the content is not related to cloud. Thus, we excluded it in this evaluation since our focus is on APT reports.

RQ2.1. Does AUTOSIGMA achieve higher coverage of described behaviors? Table III shows the number of Sigma rules generated per report. On average, ChatGPT-off-the-shelf (version: gpt-4o[10]) generates only 6 rules per report, which is often insufficient given the sophistication and multi-stage nature of APT attacks. In contrast, AUTOSIGMA, under different configurations, because its system design, specifically its attack scenario decomposition component, consistently produces a higher number of rules, around 28 for local models and 24 for Cloud-based models.

TABLE III: Number of Sigma rules generated per APT41 reports.

Report	Cloud-Based LLMs		Local-Based LLMs	Baseline
	AutoSigma Llama	AutoSigma OpenAI	AutoSigma Lily	ChatGPT off-the-shelf
Mandiant#1	18	21	25	6
Mandiant#2	14	19	22	6
Mandiant#3	30	32	36	5
Mandiant#4	25	28	30	7
TrendMicro	24	24	28	7
Avg.	22	25	28	6

[10]gpt-4o: https://developers.openai.com/api/docs/models/gpt-4o

To further validate the generated rules, we manually extracted the IoCs from the CTI reports and checked whether each IoC appears in at least one of the generated Sigma rules. Table IV reports the coverage results using different models. In a Cloud-based LLM environment, AUTOSIGMA achieves near-total coverage of IoCs; on average, it covers 95.6% and 95.2% of all IoCs reported in the underlying CTI report. For example, it covers 23 of 25 IoCs (92%) for Mandiant#1, 20/21 (95%) for Mandiant#2, and 29/32 (91%) for TrendMicro (APT41). In contrast, ChatGPT-off-the-shelf captures only a few IoCs (*e.g.,* 3/25 in Mandiant#1, 5/32 in APT41), for an average of just 17.6%. These results indicate that AUTOSIGMA fully incorporates the reported IoCs (several reports reach 100% coverage), whereas the unguided LLM misses the majority. In Local-based LLMs, AUTOSIGMA covers only about half of the IoCs, coverage ranges from 12/25 (48%) for Mandiant#1 to 8/10 (80%) for Mandiant#4, with an average of 56.0%. Appendix E provides a discussion on this gap and the difference between local and cloud-based LLM. The trend identified for APT41 holds consistently for the other APTs as well. Across APT28 and APT29, AUTOSIGMA significantly improves IoC and MITRE coverage for both cloud-based and local LLMs, while also achieving higher semantic relevancy than the ChatGPT-off-the-shelf baseline. Detailed per-APT results are reported in Appendix E.

TABLE IV: IoC-level coverage comparison between APT41 reports and generated Sigma rules.

Report	Ground Truth	Cloud-Based LLMs		Local-Based LLMs	Baseline
		AutoSigma Llama	AutoSigma OpenAI	AutoSigma Lily	ChatGPT off-the-shelf
Mandiant#1	25	23 (92%)	23 (92%)	12 (48%)	3 (12%)
Mandiant#2	21	19 (90%)	20 (95%)	9 (43%)	4 (19%)
Mandiant#3	14	14 (100%)	14 (100%)	7 (50%)	3 (21%)
Mandiant#4	10	10 (100%)	10 (100%)	8 (80%)	2 (20%)
TrendMicro	32	30 (94%)	29 (91%)	19 (59%)	5 (16%)
Avg.	–	95.2%	95.6%	56.0%	17.6%

We further assessed the technique-level coverage by extracting the set of MITRE ATT&CK techniques mentioned in each CTI report and comparing it to those appearing in the generated Sigma rules. To automate this mapping, we employ Threat2Mitre [38], which uses AI-driven language models to process natural-language threat descriptions and link them to the MITRE ATT&CK framework.

We can see in Table V that AUTOSIGMA outperforms the baseline in terms of MITRE ATT&CK coverage. With the cloud-based model, AUTOSIGMA OpenAI and AUTOSIGMA Llama achieve an average coverage of 91.2% and 90.8% of the techniques extracted from the reports, respectively, whereas ChatGPT-off-the-shelf covers only about 20.2%, due to the contextual analysis stage of the solution. Using the local LLM yields a lower but still high coverage of 70.8% for AUTOSIGMA. These percentages are derived from the per-report counts, for example, in the Mandiant#3, Threat2Mitre finds 21 techniques in the report, and AUTOSIGMA's cloud-based rules cover 19 of them when using Llama LLM, and 18 when using OpenAI LLM (90% and 86% respectively), whereas ChatGPT-off-the-shelf's rules cover only 3 (21%).

Overall, AUTOSIGMA's Sigma rules retain the vast majority of the techniques mentioned in the reports, demonstrating that the system preserves the original threat intelligence. Importantly, AUTOSIGMA also infers new additional techniques beyond those explicitly mentioned in the reports. In total, across the five APT reports, AUTOSIGMA's cloud-based rules introduce 91 new MITRE techniques (an average of 18 additional techniques per report), whereas ChatGPT-off-the-shelf adds only 12 new techniques overall. This enrichment is driven by AUTOSIGMA's use of external knowledge bases and attack decomposition. By consulting structured MITRE ATT&CK data and decomposing high-level attack behaviors into finer steps, AUTOSIGMA can infer implicit TTPs that the original report did not enumerate. This observation is also witnessed across all APTs, reported in Appendix E and Table X.

TABLE V: MITRE ATT&CK-level coverage comparison between APT41 reports and generated Sigma rules.

Report	Ground Truth	Cloud-Based LLMs						Local-Based LLMs			Baseline		
		AutoSigma Llama			AutoSigma OpenAI			AutoSigma Lily			ChatGPT off-the-shelf		
		Total	Covered	New	Total	Covered	New	Total	Covered	New	Total	Covered	New
Mandiant#1	8	16	8 (100%)	8	15	8 (100%)	7	10	6 (75%)	4	4	2 (25%)	2
Mandiant#2	12	14	10 (83%)	4	17	11 (92%)	6	9	7 (58%)	2	5	2 (17%)	3
Mandiant#3	21	29	19 (90%)	10	30	18 (86%)	12	20	14 (67%)	6	5	3 (14%)	2
Mandiant#4	13	24	13 (100%)	11	20	12 (92%)	8	16	10 (77%)	6	6	4 (31%)	2
TrendMicro	22	24	18 (81%)	6	23	19 (86%)	4	20	17 (77%)	3	5	3 (14%)	2
Avg.	–	90.8%			91.2%			70.8%			20.2%		

Moreover, we examined the relevancy of generated rules, by checking whether the generated Sigma rules reflect the semantic content of the input threat reports. To quantify this, we compute BERTScore [37] between each CTI report and its corresponding Sigma rules. BERTScore maps tokens to contextual embeddings and computes pairwise cosine similarities, yielding a score $[0-1]$ that correlates with semantic alignment. A higher BERTScore indicates that the rule's language captures more of the report's meaning.

Table VI presents the per-report BERTScores for all models for APT41. Both cloud-based versions of AUTOSIGMA achieve very similar results, with approximately the same average across reports. This shows that AUTOSIGMA is a model-agnostic solution and the variation of the underlying LLM has minimal effect on the relevancy of the generated rules. In each APT report, AUTOSIGMA's output consistently achieves the highest relevancy, outperforming the baseline. For example, in Mandiant#2, the scores are 0.778 (AUTOSIGMA OpenAI) vs 0.718 (ChatGPT-off-the-shelf). Averaging across all five reports, AUTOSIGMA attains an average BERTScore of 0.764 and 0.765, compared to 0.718 for ChatGPT-off-the-shelf.

AUTOSIGMA's generated rules include key phrases and contextual details from the report that ChatGPT-off-the-shelf sometimes omits or paraphrases less accurately. We also observed the same pattern when using a local LLM instead of a cloud-based model. Even though absolute scores are slightly lower, AUTOSIGMA using local LLM leads with a mean of 0.738 and 0.718 for ChatGPT-off-the-shelf. **RQ2.2. Are the rules produced by AUTOSIGMA syntactically valid and executable?** We define a generated Sigma rule to be *valid* if it is syntactically and operationally correct. In practice, this means the rule must be compiled without errors, and its

TABLE VI: BERTScore comparison between the input CTI report and the generated Sigma rules.

Report	Cloud-Based LLMs		Local-Based LLMs	Baseline
	AutoSigma Llama	AutoSigma OpenAI	AutoSigma Lily	ChatGPT off-the-shelf
Mandiant#1	0.766	0.762	0.722	0.709
Mandiant#2	0.779	0.778	0.748	0.718
Mandiant#3	0.749	0.752	0.723	0.701
Mandiant#4	0.771	0.775	0.745	0.729
TrendMicro	0.759	0.755	0.753	0.732
Avg.	**0.765**	**0.764**	**0.738**	**0.718**

detection logic should detect at least one relevant malicious event. Table VII shows the generated Sigma rules validity results. We can see that AUTOSIGMA cloud-based, achieves 100% validity, where every rule generated was valid (100% across all tested reports). In the local-based setting, rule validity is at 76.2% and it is lower compared to the baseline, because a lighter model is more prone to syntax slips. But, if we consider the number of valid rules that local-based AUTOSIGMA has generated, we have a larger number of rules compared to the baseline. For example, in Mandiant#4, even though the validity percentage of AUTOSIGMA is lower, it has 25 valid and ready-to-use rules, compared to the 6 valid rules generated by ChatGPT-off-the-shelf.

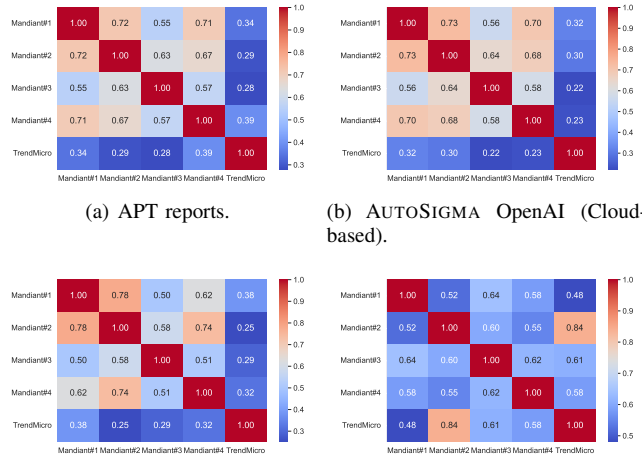
TABLE VII: Comparison of rule validity generated by different models across different APT41 reports.

Report	Cloud-Based LLMs		Local-Based LLMs	Baseline
	AutoSigma Llama	AutoSigma OpenAI	AutoSigma Lily	ChatGPT off-the-shelf
Mandiant#1	18/18 (100%)	21/21 (100%)	18/25 (72%)	6/6 (100%)
Mandiant#2	14/14 (100%)	19/19 (100%)	16/22 (73%)	6/6 (100%)
Mandiant#3	30/30 (100%)	32/32 (100%)	28/36 (78%)	5/5 (100%)
Mandiant#4	25/25 (100%)	28/28 (100%)	25/30 (83%)	6/7 (86%)
TrendMicro	24/24 (100%)	24/24 (100%)	21/28 (75%)	7/7 (100%)
Avg.	**100.0%**	**100.0%**	**76.2%**	**97.2%**

RQ2.3. Can AUTOSIGMA better adapt to unseen report structures? Adaptability measures whether AUTOSIGMA generates contextually distinct Sigma rules for different input reports, *i.e.* whether semantically different threat reports produce correspondingly different rules rather than generic hallucinations. To assess this, we compute pairwise semantic similarity using BERTScore for (i) the original APT reports, (ii) the Sigma rules generated by AUTOSIGMA, and (iii) the Sigma rules generated by ChatGPT-off-the-shelf.

Fig. 4(a) shows the semantic similarity among the original CTI reports. We see a wide range of values (≈0.28-0.72 off the diagonal), confirming that the reports differ substantially in content. Figs. 4(b)-4(d) show that AUTOSIGMA has much lower off-diagonal similarity overall (means ≈0.43-0.50) and values dropping to ≈0.17 in some cases. The generated rules from distinct reports are clearly less similar to each other, reflecting the input variability. For example, AUTOSIGMA's cloud-based configurations yield low similarity for pairs of reports that are very different (near 0.17-0.22) and higher similarity only when the original reports are themselves more alike. AUTOSIGMA using a local-based LLM, shows a similar

pattern. These results demonstrate that AUTOSIGMA's Sigma rules are contextually tailored, showing that it preserves semantic differences between reports in the output. By contrast, Fig. 4(d) shows that ChatGPT-off-the-shelf exhibits uniformly high similarities among rule outputs. Most off-diagonal entries are above ≈0.5 (mean ≈0.60, with values up to 0.84). This indicates that ChatGPT-off-the-shelf tends to generate very similar (often repetitive) Sigma rules regardless of the input report, failing to differentiate between distinct reports.



(a) APT reports. (b) AUTOSIGMA OpenAI (Cloud-based).

(c) AUTOSIGMA Lily (Local-based). (d) ChatGPT-off-the-shelf.

Fig. 4: Comparison of semantic similarities in pairs of inputs and pairs of generated Sigma rules by different models across different APT41 reports.

RQ3. How robust is AUTOSIGMA to different design choices (such as the use of LLM-as-a-Judge, model variation, and sampling temperature), and how do these choices influence the validity, relevancy, and stability of generated Sigma rules?

RQ3.1. Ablation study: What is the impact of using an LLM as an automated evaluator on rule validity and rule relevancy? In this study, we compare with/without the iterative judge loop on validity. This isolates the benefit of automatic critique-refine cycles. As shown in Table VIII, the cloud-based models equipped with the LLM-as-a-Judge mechanism achieve the highest validity, with 100% of their generated outputs being valid and directly convertible into SIEM rules. In contrast, models that do not have the LLM-as-a-Judge component exhibit lower validity ratios. For example, in the TrendMicro report, the cloud-based AUTOSIGMA (OpenAI) without the LLM-as-a-Judge generated 21 valid rules out of 24 total rules, whereas using LLM-as-a-Judge increases this to 100%. In the local setting, a similar trend is observed: the version without LLM-as-a-Judge achieves an average validity of 47.5% across all APTs, while integrating the Judge increases this to 71.7%, representing an improvement of 24.2%.

RQ3.2. Design choice study: How does the sampling temperature influence the balance between rule diversity and hallucination frequency? Hallucination is a major challenge in LLM-based solutions. Although AUTOSIGMA uses

TABLE VIII: Comparison of rule validity by different models across different APT41 reports.

Report	Cloud-Based LLMs			Local-Based LLMs		
	AutoSigma Llama (w/ Judge)	AutoSigma Llama (w/o Judge)	AutoSigma OpenAI (w/ Judge)	AutoSigma OpenAI (w/o Judge)	AutoSigma Lily (w/ Judge)	AutoSigma Lily (w/o Judge)
Mandiant#1	18/18 (100%)	16/18 (89%)	21/21 (100%)	18/21 (86%)	18/25 (72%)	13/25 (52%)
Mandiant#2	14/14 (100%)	12/14 (86%)	19/19 (100%)	17/19 (89%)	16/22 (73%)	10/22 (45%)
Mandiant#3	30/30 (100%)	24/30 (80%)	32/32 (100%)	26/32 (81%)	28/36 (78%)	20/36 (56%)
Mandiant#4	25/25 (100%)	21/25 (84%)	28/28 (100%)	26/28 (93%)	25/30 (83%)	16/30 (53%)
TrendMicro	24/24 (100%)	21/24 (88%)	24/24 (100%)	21/24 (88%)	21/28 (75%)	12/28 (43%)
Avg.	**100.0%**	**85.4%**	**100.0%**	**87.4%**	**76.2%**	**49.8%**

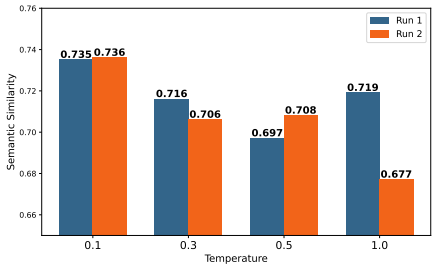


Fig. 5: Effect of temperature on semantic similarity.

LLM-as-a-Judge to mitigate the risk of hallucination, there is currently no standardized metric for quantifying hallucinated content in this context. Therefore, we analyze behaviors that affect the likelihood of hallucination and present the measures taken in AUTOSIGMA to reduce this likelihood.

Temperature [39] is a key controllable parameter in LLM sampling that controls randomness and diversity. Higher temperatures tend to increase linguistic variation and "creativity" but also introduce instability and hallucinated logic. To evaluate its impact, we conducted an experiment using one representative APT report and ran AUTOSIGMA twice per temperature value $T = \{0.1, 0.3, 0.5, 1.0\}$. For each run, we computed the semantic similarity using BERTScore between the generated rules and the original CTI report. As shown in Fig. 5, the outputs at $T = 0.1$ remain consistent across runs, while higher temperatures cause semantic divergence. This confirms that maintaining a low temperature (0.1 in our main experiments) provides stable, low-variance results and effectively suppresses hallucinated content.

RQ3.3. Does domain-specific fine-tuning reduce hallucination and improve coverage compared to prompt-only generation? Fine-tuning the base LLM with domain-specific data can further improve the factual grounding and consistency of generated rules. To validate this, we compared two variants of AUTOSIGMA using the same temperature setting ($T = 0.1$): one with the cybersecurity fine-tuned model *Lily-Cybersecurity-7B-v0.2* and another using a generic Mistral LLM. As shown in Fig. 6, the fine-tuned Lily model achieves consistently higher semantic similarity across runs, confirming its reduced hallucination behavior. This observation aligns with prior work such as CyberPalAI [40].

RQ3.4. How does AUTOSIGMA perform with noisy data? Although our framework assumes that CTI reports are reliable and that CTI poisoning or manipulation is out of scope, in practice external CTI should be considered untrusted. To assess the robustness of AUTOSIGMA under degraded input conditions, we conduct an experiment where

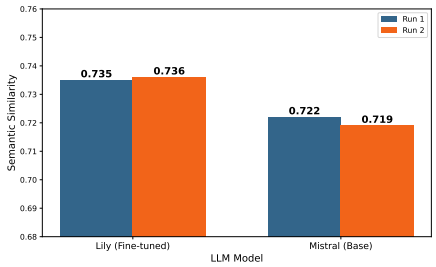


Fig. 6: Effect of model fine-tuning on hallucination.

controlled Gaussian noise [41] is injected into the CTI reports at different contamination rates. The noise is applied at the text level using character-level perturbations, and the resulting outputs are compared against those generated from the original clean reports using similarity metrics. The results

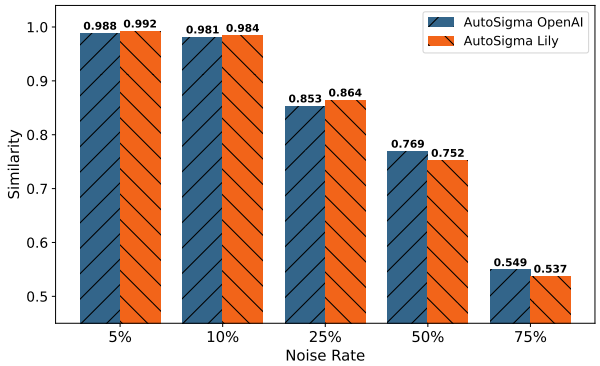


Fig. 7: Effect of noisy data.

are shown in Fig. 7. As expected, the similarity decreases as the contamination rate increases. Up to a 10% contamination rate, AUTOSIGMA maintains a high similarity above 0.98, indicating strong robustness to minor perturbations. At 25% noise, the similarity drops to approximately 0.85, reflecting moderate degradation. Beyond this point, the performance declines more significantly, reaching 0.75 at 50% and 0.53 at 75% contamination. These results demonstrate that while AUTOSIGMA is resilient to low levels of noise, substantial corruption in the input can negatively impact the quality of the generated rules. It is important to note that AUTOSIGMA is designed to operate on validated CTI reports typically used by SOC teams. Handling adversarial manipulation or heavily corrupted CTI inputs is beyond the scope of this work and is left for future investigation.

VIII. DISCUSSION

The evaluation of AUTOSIGMA highlights its strong performance in generating high-quality, deployable Sigma rules. However, beyond reporting numerical metrics, it is essential to critically assess the solution's design and identify areas where improvements can be made. A promising direction for enhancing robustness involves combining generative reasoning with rule-based validation logic, such as leveraging structured grammars, detection logic templates, and constraints informed by known adversary behaviors. This would enable more principled rule synthesis that does not depend solely on LLM fluency. Additionally, the current rule generation loop is generative at every iteration. Moving toward a guided refinement process, where intermediate rule candidates are evaluated not only by another model but also by static analyzers or predefined criteria, could reduce hallucinations and enforce stronger alignment with security semantics. A separate concern is the execution time of the system. Because AUTOSIGMA operates as a multi-stage pipeline with enrichment, template retrieval, and iterative validation, the overall latency can be non-trivial. While this is acceptable for offline threat hunting use cases, where rules are generated as part of periodic analysis, it becomes a limitation in time-sensitive environments. Although the pipeline was not designed for real-time deployment—it is designed for offline deployment—improving the efficiency of the validation loop and introducing caching or early-exit strategies could significantly reduce processing delays and make the system more responsive. Despite these performance trade-offs, it is worth emphasizing that the rule synthesis process only needs to run once per report and does not interfere with real-time alert handling, making this overhead manageable in most hunting workflows. A further limitation relates to the training data of the underlying large language models. For cloud-based models such as OpenAI GPT-4o and LLaMA, the exact composition of the training data is not publicly disclosed, and therefore we cannot guarantee that the CTI reports used in our evaluation were not part of their pretraining data. While these models are general-purpose and not specifically trained for CTI-to-Sigma rule generation tasks, the possibility of prior exposure cannot be fully ruled out. Similarly, for the local model (Lily), although it is fine-tuned on cybersecurity data, the exact sources of this data are not fully specified. This limitation is inherent to current LLM-based systems and should be considered when interpreting the results.

IX. CONCLUSION

In this paper, we introduced AUTOSIGMA, a fully automated framework for generating actionable Sigma rules from unstructured CTI reports. Through the integration of knowledge-based enrichment, template-guided rule generation, and a structured multi-phase pipeline, AUTOSIGMA produces contextually relevant and semantically accurate Sigma rules that can be used for hunting activities. By leveraging external threat knowledge, aligning with existing detection templates, and enforcing rule correctness through iterative validation, AUTOSIGMA offers a robust and effective mechanism for streamlining threat detection and enhancing security workflows. Our evaluation using public CTI reports demonstrated that AUTOSIGMA achieves high validity in rule generation, consistently outperforming baseline models, and requiring minimal manual intervention. Future work will involve validating AUTOSIGMA across broader, more heterogeneous datasets and in operational environments involving live alert streams.

ACKNOWLEDGMENTS

The authors used Generative AI (*i.e.* GPT-4o) solely for grammar and language refinement.

REFERENCES

[1] Mandiant, "Double Dragon APT41, a dual espionage and cyber crime operation," https://www.mandiant.com/sites/default/files/2022-02/rt-apt41-dual-operation.pdf, 2022.

[2] CrowdStrike, "Global Threat Report," https://www.crowdstrike.com/global-threat-report/, 2024.
[3] M. Suominen *et al.*, "Cyber threat intelligence management: Tools, techniques, and best practices," *Cybersecurity Science*, 2024.
[4] G. Husari *et al.*, "Ttpdrill: Extracting threat techniques and tactics from cyber threat reports," *Journal of Cyber Threat Intelligence*, 2017.
[5] L. Gao *et al.*, "Threatraptor: Extracting indicators of compromise and threat behavior from cti reports," in *IEEE Symposium on Security and Privacy*, 2021, pp. 300–315.
[6] T. Alam *et al.*, "LADDER: Automated Extraction of Attack Patterns from CTI Reports," in *Proceedings of the ACM Conference on Security and Privacy*, 2024, pp. 250–265.
[7] W. Zhang *et al.*, "A systematic review of large language models in cybersecurity," *ACM Computing Surveys*, 2024.
[8] M. A. Ferrag *et al.*, "Generative ai and large language models for cybersecurity applications," *IEEE Transactions on Cybersecurity*, 2024.
[9] W. P. M. III *et al.*, "An Interview Study on Third-Party Cyber Threat Hunting Processes in the U.S. Department of Homeland Security," in *USENIX Security*, 2024.
[10] Y. Schwartz *et al.*, "Llmcloudhunter: Harnessing llms for automated extraction of detection rules from cloud-based cti," in *Proceedings of the ACM on Web Conference*, 2025.
[11] M. Xu *et al.*, "Intelex: A llm-driven attack-level threat intelligence extraction framework," https://arxiv.org/abs/2412.10872, 2024.
[12] S. Alaeifar *et al.*, "Current challenges and future directions in cyber threat intelligence sharing," *Journal of Cybersecurity Research*, 2024.
[13] N. Satvat *et al.*, "Extractor: A framework for extracting structured threat intelligence from cti reports," *Cybersecurity Intelligence Review*, 2021.
[14] C. Glyer *et al.*, "This Is Not a Test: APT41 Initiates Global Intrusion Campaign Using Multiple Exploits," https://www.mandiant.com/resources/blog/apt41-initiates-global-intrusion-campaign-using-multiple-exploits, 2024.
[15] T. Satyapanich *et al.*, "Casie: Extracting cybersecurity event information from text," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.
[16] I. Sarhan *et al.*, "Open-cykg: An open cyber threat intelligence knowledge graph," *Knowledge-Based Systems*, 2021.
[17] J. Park *et al.*, "SecIE: A Full-Stack Information Extraction System for Cybersecurity Intelligence," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2022.
[18] J. Liu *et al.*, "Tricti: an actionable cyber threat intelligence discovery system via trigger-enhanced neural network," *Cybersecurity*, 2022.
[19] S. Shah *et al.*, "Cyberentrel: Joint extraction of cyber entities and relations using deep learning," *Computers & Security*, 2024.
[20] F. Ahmadou *et al.*, "Automated attack testflow extraction from cyber threat report using bert for contextual analysis," https://arxiv.org/abs/2507.07244, 2025.
[21] M. D. Purba *et al.*, "Extracting actionable cyber threat intelligence from twitter stream," in *IEEE International Conference on Cyber Security and Resilience (CSR)*, 2023.
[22] N. Rastogi *et al.*, "Actionable cyber threat intelligence using knowledge graphs and large language models," *arXiv preprint arXiv:2407.02528*, 2023.
[23] X. Liu *et al.*, "A study on chatgpt for industry 4.0: Background, potentials, and preliminary applications," *Journal of Industrial Information Integration*, 2023.
[24] W. Zhang *et al.*, "Few-shot learning of ttps classification using large language models," 2023.
[25] Fengrui *et al.*, "Few-shot learning of ttps classification using large language models," *arXiv preprint arXiv:2401.0372*, 2024.
[26] P. A. Gandhi *et al.*, "Shield: Apt detection and intelligent explanation using llm," *arXiv preprint arXiv:2502.02342*, 2025.
[27] B. H. S. Durga *et al.*, "Information extraction from text messages using natural language processing," in *ICCCI*, 2023.
[28] H. Brådland *et al.*, "A New HOPE: Domain-agnostic Automatic Evaluation of Text Chunking," *arXiv 2505.02171*, 2025.
[29] L.-H. Lin *et al.*, "Attack tactic identification by transfer learning of language model," *arXiv 2209.00263*, 2022.
[30] A. Basel *et al.*, "Smet: Semantic mapping of cve to att&ck and its application to cybersecurity," in *Data and Applications Security and Privacy XXXVII*, 2023.
[31] L. Li *et al.*, "Automated discovery and mapping att&ck tactics and techniques for unstructured cyber threat intelligence," *Computers & Security*, 2024.
[32] J. Devlin *et al.*, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2019.
[33] CVE Program, "Cve: Common vulnerabilities and exposures," https://www.cve.org/, 2025.
[34] T. Zhang *et al.*, "Bertscore: Evaluating text generation with bert," *arXiv 1904.09675*, 2020.
[35] K. Papineni *et al.*, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*, 2002.
[36] J. Gu *et al.*, "A survey on llm-as-a-judge," https://arxiv.org/abs/2411.15594, 2025.
[37] T. Zhang *et al.*, "Bertscore: Evaluating text generation with bert," in *International Conference on Learning Representations (ICLR)*, 2020.
[38] L. Y. Cheng, "Threats To MITRE (CWD, ATT&CK) AILLM Mapper," https://github.com/LiuYuancheng/Threats_2_MITRE_AI_Mapper, 2024.
[39] A. Holtzman *et al.*, "The curious case of neural text degeneration," https://arxiv.org/abs/1904.09751, 2020.
[40] M. Levi *et al.*, "Cyberpal.ai: Empowering llms with expert-driven cybersecurity instructions," https://arxiv.org/abs/2408.09304, 2024.
[41] C. M. Bishop *et al.*, *Pattern recognition and machine learning*. Springer, 2006.
[42] A. Pennino *et al.*, "Game over: Detecting and stopping an apt41 operation," https://cloud.google.com/blog/topics/threat-intelligence/game-over-detecting-and-stopping-an-apt41-operation, 2023.
[43] G. Cloud, "Apt41 initiates global intrusion campaign using multiple exploits," https://cloud.google.com/blog/topics/threat-intelligence/apt41-initiates-global-intrusion-campaign-using-multiple-exploits/, 2024.
[44] G. Cloud, "Apt41 us state governments: Threat intelligence insights," https://cloud.google.com/blog/topics/threat-intelligence/apt41-us-state-governments/?hl=en, 2023.
[45] H. Hiroaki *et al.*, "Earth Baku An APT Group Targeting Indo-Pacific Countries With New Stealth Loaders and Backdoor," https://documents.trendmicro.com/assets/white_papers/wp-earth-baku-an-apt-group-targeting-indo-pacific-countries.pdf, 2021.
[46] ANSSI (CERT-FR), "Campagnes d'attaques du mode opératoire APT28 depuis 2021," https://cert.ssi.gouv.fr/uploads/CERTFR-2023-CTI-009.pdf, 2023.
[47] ClearSky Cyber Security Research Team, "Doppelgänger NG: Russian Cyberwarfare campaign," https://www.clearskysec.com/wp-content/uploads/2024/02/DoppelgangerNG_ClearSky.pdf, 2024.
[48] Insikt Group (Recorded Future), "BlueDelta Exploits Ukrainian Government Roundcube Mail Servers to Support Espionage Activities," https://go.recordedfuture.com/hubfs/reports/cta-2023-0620.pdf, 2023.
[49] N. Khadgi *et al.*, "APT28: Inside Forest Blizzard's New Arsenal," https://www.logpoint.com/wp-content/uploads/2024/06/logpoint-etpr-forest-blizzard.pdf, 2024.
[50] Maverits Cybersecurity Center, "APT28, the long hand of Russian interests," https://www.maverits.com/post/apt28-the-long-hand-of-russian-interests, 2023.
[51] CrowdStrike Intelligence Team, "Observations from the stellarparticle campaign," https://www.crowdstrike.com/blog/observations-from-the-stellarparticle-campaign/, 2023.
[52] National Coordination Center for Cybersecurity at the NSDC of Ukraine, "APT29 attacks Embassies using CVE-2023-38831," https://rnbo.gov.ua/files/2023_YEAR/CYBERCENTER/november/APT29attacksEmbassiesusingCVE-2023-38831-reporten.pdf, 2023.
[53] Recorded Future Insikt Group, "Bluebravo adapts to target diplomatic entities with graphicalproton malware," https://www.recordedfuture.com/research/bluebravo-adapts-to-target-diplomatic-entities-with-graphicalproton-malware, 2024.
[54] Mandiant- Google Cloud, "Not so cozy: An uncomfortable examination of a suspected apt29 phishing campaign," https://www.fireeye.com/blog/threat-research/2018/11/not-so-cozy-an-uncomfortable-examination-of-a-suspected-apt29-phishing-campaign.html, 2018.
[55] "Russian GRU Conducting Global Brute Force Campaign to Compromise Enterprise and Cloud Environments," https://media.defense.gov/2021/Jul/01/2002753896/-1/-1/0/CSA_GRU_GLOBAL_BRUTE_FORCE_CAMPAIGN_UOO158036-21.PDF, 2021.

APPENDIX A
EXAMPLE OF OUTPUTS

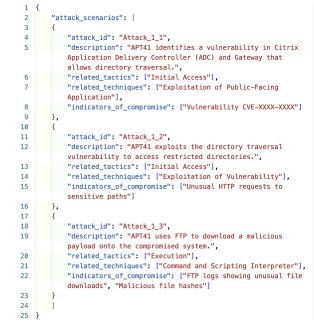
Fig. 8 shows an example of attack scenario enrichment and attack step decomposition. In Fig. 8(a), we observe the output of the enrichment phase, which includes extracted entities such as the CVE (lines 9-17), threat actor (lines 19-24), and

contextual description. In Fig. 8(b), the enriched scenario is decomposed into three smaller attack steps: the first involves the identification of a vulnerability in Citrix ADC (Initial Access), the second describes the actual exploitation attempt using that vulnerability, and the third captures the follow-up action of downloading a malicious payload using FTP (Execution phase). Each of these steps is now ready to be handled separately by the rule generation module.



```
{
  "attack_id": "Attack_1",
  "description": "APT41 exploited CVE-2019-19781 to gain initial access to the target system by utilizing FTP to download a malicious payload.",
  "related_tactics": ["Initial Access"],
  "related_techniques": ["Exploitation of Remote Services"],
  "indicators_of_compromise": ["CVE-2019-19781", "66.42.98[.]220"],
  "cve_enrichment": [
    {
      "cve_details": {
        "cve_id": "CVE-2019-19781",
        "description": "An issue was discovered in Citrix Application Delivery Controller (ADC) and Gateway 10.5, 11.1, 12.0, 12.1, and 13.0. They allow Directory Traversal.",
        "cvss_score": 7.5,
        "severity": "High",
        "published_date": "2019-12-27",
        "last_modified": "2025-02-04"
      }
    }
  ],
  "threat_actor_enrichment":
    {
      "apt_group": "APT41",
      "mitre_url": "https://attack.mitre.org/groups/G0092/",
      "status": "Success"
    }
}
```


(a) An example of attack scenario enrichment output.



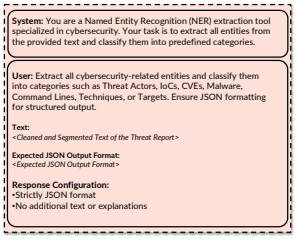
```
{
  "attack_scenarios": [
    {
      "attack_id": "Attack_1_1",
      "description": "APT41 identifies a vulnerability in Citrix Application Delivery Controller (ADC) and Gateway that allows directory traversal.",
      "related_tactics": ["Initial Access"],
      "related_techniques": ["Exploitation of Public-Facing Application"],
      "indicators_of_compromise": ["Vulnerability CVE-XXXX-XXXX"]
    },
    {
      "attack_id": "Attack_1_2",
      "description": "APT41 exploits the directory traversal vulnerability to access restricted directories.",
      "related_tactics": ["Initial Access"],
      "related_techniques": ["Exploitation of Vulnerability"],
      "indicators_of_compromise": ["Unusual HTTP requests to sensitive paths"]
    },
    {
      "attack_id": "Attack_1_3",
      "description": "APT41 uses FTP to download a malicious payload onto the compromised system.",
      "related_tactics": ["Execution"],
      "related_techniques": ["Command and Scripting Interpreter"],
      "indicators_of_compromise": ["FTP logs showing unusual file downloads", "Malicious file hashes"]
    }
  ]
}
```


(b) An example of attack step decomposition output.

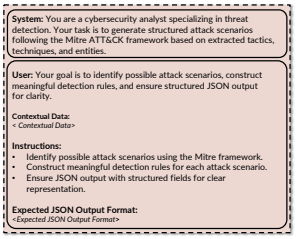
Fig. 8: Outputs of attack scenario enrichment and step decomposition.

Appendix B
LLM Prompting

Basic prompts such as "extract NERs from a given text" proved inadequate for cybersecurity tasks, which demand higher specificity and structure. To address this, we employed prompt engineering techniques that included explicit task instructions, relevant examples, and clearly defined output formats. An illustration of our structured prompt engineering is shown in Fig. 9.



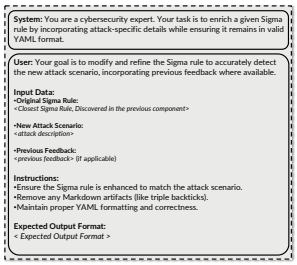
(a) NER Extractor Component.



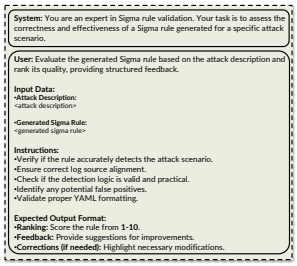
(b) Attack Scenario Extractor Component.

Fig. 9: Example of prompts used by AutoSigma.

LLM-as-a-Judge Loop: To ensure the quality of the generated Sigma rules, AutoSigma employs an LLM-as-a-Judge loop in which a generator and a validator model work iteratively. The generator produces an initial Sigma rule (as shown in Fig. 10(a)), a separate LLM, used as the validator, then evaluates the generated rule against predefined criteria, including correctness, completeness, and Sigma syntax compliance (Fig. 10(b)). If the validator's score falls below a fixed threshold (8/10 across all dimensions), its feedback is returned to the generator, and the refinement loop continues until the quality criteria are met. Furthermore, as illustrated



(a) Prompt used for Sigma rule generator.



(b) Prompt used for Sigma rule validator.

Fig. 10: LLM-as-a-Judge prompts.

in Fig. 10, the prompts used for LLM-as-a-Judge are adaptive and dynamically constructed.

Appendix C
Proof of Concept (PoC)

Fig. 11 presents the Proof of Concept interface we developed for AutoSigma, highlighting the end-to-end workflow of the AutoSigma. The UI is structured into four main sections, visually indicated by blue rounded-square icons:

1. **Input Area:** Users select the LLM model (local or cloud-based) and upload or paste the threat report to initiate analysis.
2. **Knowledge Extraction & Similar Rule Discovery:** The left panel visualizes key statistics such as the total number of extracted Named Entities (NERs), MITRE ATT&CK techniques, tactics, and attack tests. The right panel displays a bubble chart summarizing similar Sigma rules identified from the SigmaHQ repository.
3. **Contextual Information:** This area shows the detailed breakdown of contextual information extracted by AutoSigma, including recognized NERs, ATT&CK techniques, tactics, and the reconstructed attack scenario.
4. **Outputs and Rules:** The final output section displays the generated Sigma detection rules and allows users to review or export them.

Appendix D
CTI Reports Summary

Table IX provides a summary of the used CTI reports per APT in our evaluation. To reflect the diversity and realism of operational CTI, we selected reports from multiple security vendors, written at different levels of technical depth, and varying significantly in both length and structure. These reports mix multiple forms of information, including high-level executive summaries, detailed technical analyses, command line snippets, screenshots, figures, and narrative descriptions of attacker behavior. Using such a varied corpus demonstrates that AutoSigma can operate effectively across heterogeneous and unstructured report formats, regardless of whether the content is tactical, technical, or strategic.

Appendix E
Local LLMs Versus Cloud LLMs

While our evaluation compared both cloud-based and local LLM deployments, it is important to reiterate that AutoSigma was designed as a pipeline, not as a benchmark for individual LLMs. The performance gaps we observed between

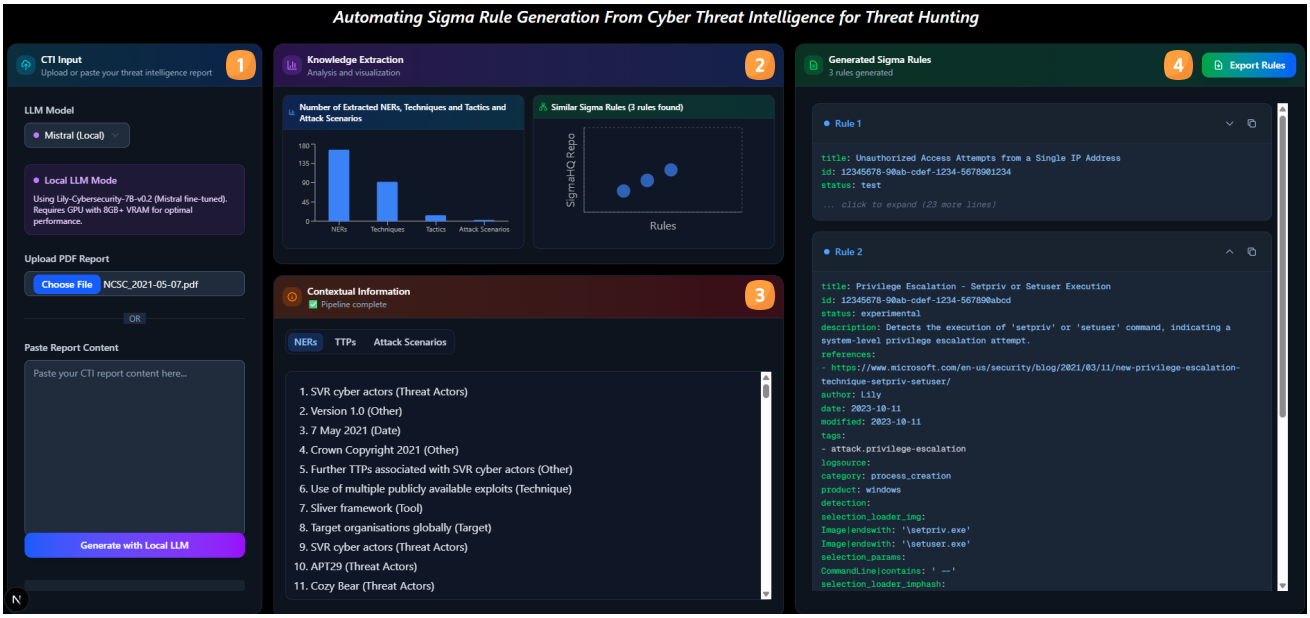


Fig. 11: Screenshot of the AUTOSIGMA PoC interface.

TABLE IX: Summary of CTI reports used in the evaluation.

Report	Target Industries	Main TTPs	Detected	Published	Pages
APT41					
Mandiant#1 [1]	Banking , Finance, Government, High-tech	PowerShell usage, Cobalt deployment, FTP for payload delivery	Jan-Mar 2020	2020-03-25	13
Mandiant#2 [42]	Gaming, Healthcare, High-tech	Exploiting vulnerabilities, Web shells, Backdoors	Apr 2019	2019-08-19	9
Mandiant#3 [43]	Telecoms, News/Media, Software firms	Backdoors, Reconnaissance, Credential theft	Since 2012	2019-08-07	68
Mandiant#4 [44]	U.S. State Gov, Insurance, Telecom	SQL injection, Zero-day exploits, DNS manipulation	May 2021-Feb 2022	2022-03-08	9
TrendMicro [45]	Enterprises, Government, Airline, Publishing	StealthVector, Backdoors, DNS manipulation	Jul 2020	2021-08-24	55
APT28					
ANSSI [46]	Gov., universities, think tanks (France)	CVE-2023-23397, phishing, compromised edge devices	H2 2021-2023	2023-10-26	7
ClearSky [47]	Media, public info outlets (US, EU, Israel)	Disinformation, fake news infra reuse	2023-2024	2024-02-22	29
Insikt#1 [48]	Ukrainian government	Roundcube/Outlook phishing (CVE-2023-23397), credential theft	2021-2023	2023-06-20	17
Logpoint [49]	NATO-aligned gov./defense/media	GooseEgg exploit, Print Spooler abuse, LOLBINs	2023-2024	2024-04-05	29
Maverits [50]	Gov., foreign affairs (Ukraine, Europe)	Backdoors, LOLBINs, cybercrime collaboration	2022-2023	2023-01-27	32
APT29					
CrowdStrike [51]	Foreign ministries, NATO governments	Credential phishing, malware loader	2021-2022	2022-01-27	21
HKUK [52]	Multiple UK sectors incl. academia	Password spraying, VPN compromise, supply chain	2021-2023	2023-11-14	12
Insikt [53]	Think tanks, foreign policy institutions	Credential theft via phishing themes	2022-2023	2023-07-27	19
Mandiant#5 [54]	U.S. and European gov. orgs	Credential theft, C2 tunneling, phishing kits	2016-2018	2018-11-19	17
NCSC [55]	Defense, law, gov. entities	Credential spraying, Exchange exploit	2019-2021	2021-05-07	7

the two settings reflect differences in model capabilities at specific stages of the pipeline rather than shortcomings in the design. In fact, our results suggest that each class of model brings unique strengths to different parts of the system. The local LLM used in our experiments , a fine-tuned variant of Mistral with a cybersecurity-specific training corpus , performed particularly well during the early analytical stages. Tasks such as NER and TTP extraction, and attack scenario extraction benefited from the model's domain alignment. In these steps, the local model demonstrated strong precision and contextual understanding, likely due to the security-specific patterns it had learned during fine-tuning. However, the final stages of the pipeline, especially the generation of Sigma rules, placed greater demands on generative capacity, instruction adherence, and formatting consistency. Here, the cloud-based models exhibited a clear advantage. Backed by significantly larger parameter counts and broader pretraining data, these cloud-based models consistently produced more well-formed, coherent, and complete Sigma rules. Because our validation and scoring processes take place after generation, the strength

of the initial drafts plays a decisive role in final outcomes. This explains why, even though the local model handled upstream analysis well, the API models slightly outperformed it in downstream metrics like rule validity and semantic alignment. This analysis also clarifies that the observed performance delta is not due to the pipeline failing on local models, but to generative limitations that the generator and validator alone cannot fully overcome. It also opens a path for hybrid approaches, such as using a local model for extraction and an API model for generation, offering future work opportunities to balance performance and deployment constraints more effectively. It is also worth noting that, as discussed earlier, cloud-based LLMs introduce a small operational cost, while removing the need for computational resources. In our experiments, the cloud-based versions of AUTOSIGMA had an estimated cost of $0.0015 - $0.003 per run, depending on the number of iterations the LLM-as-a-Judge loop takes. Since these models require no dedicated GPU resources or local model hosting, this makes them convenient for lightweight deployment. Conversely, local LLMs are free to use, yet demand sufficient hardware capacity, typically GPUs, to deploy. These trade-offs highlight an important design consideration for practitioners.

APPENDIX F
LLM SPECIFICATIONS

For our experiments, we used three LLMs across both cloud-based and local configurations, selected to capture variations in architecture, parameter scale, and domain specialization. All models were executed with a fixed temperature of 0.1 and a maximum generation length of 512 tokens to ensure deterministic and reproducible outputs.

- **GPT-4o-mini**: a general-purpose LLM designed for tasks such as reasoning, summarization, and text generation. It serves as a representative commercial closed-weight model.
- **Llama 3.3-70B**: an open-weight, large-scale model with 70 billion parameters, capable of multi-turn reasoning and contextual comprehension. It is used to assess performance consistency across model families under identical prompting conditions.
- **Lily-Cybersecurity-7B-v0.2**: a fine-tuned version of Mistral-7B adapted to the cybersecurity domain, with 22,000 hand-crafted cybersecurity and hacking-related data pairs. The dataset was then run through an LLM to provide additional context, personality, and styling to the outputs.

APPENDIX G
AVERAGE EVALUATION ACROSS MULTIPLE APTS

As shown in Table X, the trend observed for APT41 holds consistently for APT28 and APT29 as well. AUTOSIGMA significantly improves IoC coverage using both cloud-based and local LLMs. With cloud-based LLMs, the average IoC coverage increases from 20.1% (ChatGPT-off-the-shelf baseline) to 90.4%. Similarly, in the local LLM setting, AUTOSIGMA improves the average IoC coverage from 20.1% to 60.7%, corresponding to an increase of 40.6 percentage points. This observation is also consistent with our MITRE ATT&CK coverage results (see Table V in the main text), demonstrating that AUTOSIGMA not only matches the report-derived technique coverage, but also significantly broadens it, yielding a more comprehensive Sigma rule set that covers more techniques across different APT campaigns. The same pattern appears when considering semantic relevancy. For complex reports in APT28 and APT29, the average relevancy of AUTOSIGMA is high (*i.e.* 0.816 and 0.814), while ChatGPT-off-the-shelf lags behind (*i.e.* 0.711 and 0.690). This confirms that the rules generated by AUTOSIGMA capture the critical context and tactics of each campaign rather than producing generic or poorly aligned detections.

TABLE X: Average results of all APTs across models.

APT	Model	IoC Coverage	MITRE Coverage	Relevancy	Validity
APT41	AUTOSIGMA Llama (Cloud-based)	95.2	90.8	76.5	100
	AUTOSIGMA OpenAI (Cloud-based)	95.6	91.2	76.4	100
	AUTOSIGMA Lily (Local-based)	56.0	70.8	73.8	76
	ChatGPT-off-the-shelf	17.6	20.2	71.8	97
APT29	AUTOSIGMA Llama (Cloud-based)	86.0	91.6	81.6	100
	AUTOSIGMA OpenAI (Cloud-based)	87.0	91.6	81.4	100
	AUTOSIGMA Lily (Local-based)	62.4	63.6	76.1	75
	ChatGPT-off-the-shelf	28.0	29.4	69.0	93
APT28	AUTOSIGMA Llama (Cloud-based)	89.5	92.9	81.4	100
	AUTOSIGMA OpenAI (Cloud-based)	88.6	91.8	81.6	100
	AUTOSIGMA Lily (Local-based)	63.8	63.4	75.1	63
	ChatGPT-off-the-shelf	14.8	25.8	71.1	87
Avg.	AUTOSIGMA Llama (Cloud-based)	90.2	91.7	79.8	100
	AUTOSIGMA OpenAI (Cloud-based)	90.4	91.5	79.8	100
	AUTOSIGMA Lily (Local-based)	60.7	65.9	75.0	71.7
	ChatGPT-off-the-shelf	20.1	25.1	70.6	92.3